

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Abhängige Objekte, Container und Iteratoren

Wärmespeicherintegration

April 2026, Version: 3.2

1 Einführung

Diese Aufgabe setzt die Entwicklung des Wärmespeichers auf Basis der in der letzten Aufgabe entwickelten Einzelkomponenten fort. Dabei geht es in dieser Aufgabe im Wesentlichen um das Zusammenwirken unterschiedlicher Objekte.

Bei einer realistischen Anlage, sind die Kessel und zugehörigen Brenner weniger als Einzelgeräte zu sehen, sondern als kombinierte Einheit eines beheizbaren Kessels. Diese Tatsache soll nun auch im Programmdesign umgesetzt werden. Zudem wird in dieser Aufgabe die noch fehlende Mausbedienung nachgerüstet.

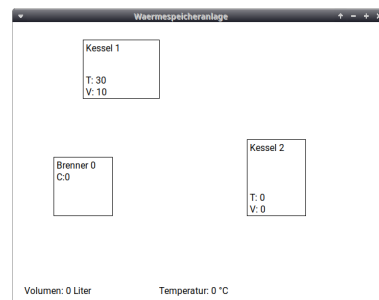


Abbildung 1: Ausgangsposition nach Abschluss der vorangehenden Aufgabe: **Boiler** und **Burner** können unabhängig voneinander instanziiert und angesprochen werden.

Hinweis: Lesen Sie sorgfältig!

Hinweis: Testen Sie unbedingt wie angegeben nach jeden Entwicklungsschritt Ihr Programm!

Hinweis: Basteln Sie nicht herum, sondern bemühen Sie sich um eine überlegte und strukturierte Vorgehensweise. Holen Sie uns zu Hilfe, wenn die Lage ausweglos erscheint!

Sie ersparen sich damit enorm viel Zeit!

Vorbereitung

- ☐ ☕ **this-Pointer:** Lesen Sie nach, was **this**-Pointer sind und wie sie korrekt verwendet werden. 6
- ☐ ☕ **Copy-Konstruktoren:** Lesen Sie nach, was Copy-Konstrukturen sind, wann sie aufgerufen werden und woran man sie erkennen kann. Welche wichtige Aufgabe müssen sie erfüllen und worauf ist dabei zu achten? 6

- ☐ ☕☕☕ **Zusammenhang zwischen Copy-Konstruktoren, Zuweisungsoperatoren und Destruktoren:** Diese „Gang of Three“ tritt fast immer gemeinsam in einer Klassenimplementierung auf. Ergründen Sie den funktionalen Zusammenhang zwischen den dreien um zu verstehen, warum dies so ist. Klären Sie dabei aber auch die Unterschiede der drei Gangmitglieder! 7
- ☐ ☕☕☕ **Iteratoren: Fähigkeiten und Funktion:** Lesen Sie nach, was Iteratoren sind bzw. was sie können. Eine knappe Einführung finden Sie z.B. im Skript. Das Ziel ist, eine Iterator-Klasse selbst zu implementieren und zu verwenden. 13

2 Projekt vorbereiten

Für diese Aufgabe gibt es **keine neuen Dateien** oder Verzeichnisse im Upstream-Projekt. Sie **arbeiten am bereits bestehenden Kessel-Projekt weiter und ergänzen** es um neue Implementierungen.

Um dennoch den Stand der fertiggestellten Aufgabe Kessel problemlos wiederherstellen zu können, wird der entsprechende **Softwarestand gekennzeichnet**.



Stellen Sie sicher, dass der aktuelle Entwicklungszustand mit einem Commit gesichert wurde. Falls nicht, holen Sie dies jetzt nach.

Bevor mit der weiteren Entwicklungsarbeit begonnen wird, „merken“ Sie sich den aktuellen Zustand Ihres Entwicklungsprojekts mit einem sogenannten „Tag“¹.



Markieren Sie den aktuellen Entwicklungszustand durch setzen eines sog. „Tags“.



Erzeugen Sie für die aktuelle Version ein sogenanntes „Tag“, indem Sie in `gitk` einen Rechtsklick auf die aktuelle Version ausführen und *Markierung erstellen* auswählen. Verwenden Sie `Einzelkomponenten_fertiggestellt` als Markierung. Alternativ können Sie natürlich auch in der `git - bash` das Kommando

```
git tag "Komponenten_fertiggestellt"
```

ausführen.

Laden Sie danach Ihr Projekt auf den Server, und aktivieren Sie dabei in `git gui` die Option „Markierungen übertragen“ bzw. alternativ bei Verwendung der Kommandozeile

```
git push --tags
```

Die Markierung wird in der graphischen Darstellung durch eine Art „Fähnchen“ an der Version gekennzeichnet und erlaubt Ihnen das spätere Auffinden der entsprechenden Version.

¹von Englisch „tag“: Schild

Prüfen Sie, ob das „Tag“ korrekt in der Liste der Tags in Ihrem Projekt auf dem gitlab-Server auftaucht.

3 Erweiterung der Klasse Burner

Um die Bedienung der Simulation zu erleichtern, soll sie mit einer Mausbedienung ausgestattet werden.

Die Heizung der einzelnen Brenner soll über einen Mausklick auf den Brenner (oder bei verbundenen Brennern) auf den Kessel oder den Brenner ermöglicht werden. Um zu prüfen, ob ein Mausklick ein Objekt auf dem Bildschirm „getroffen“ hat, muss geprüft werden, ob die Position, an die geklickt wurde, innerhalb eines der Objekte der Anlage lag und ggf. in welchem.

Eine naheliegende Lösung ist es, diese Prüfung die jeweiligen Objekte selbst durchführen zu lassen, da sie ihre Position und Form kennen und dementsprechend auch diese Prüfung vornehmen können.

Burner
+ Name : string + Position : Point + MyBoiler : Boiler* + ConsumedFuel : double + ActualDimension : Point + ActualPosition : Point
+ Burner(MyBoiler : Boiler*, Name : string&, Position Point) + feedFuel(double Amount) : void + show() : void + updateCoordinates() : void + contains() : bool

Hierfür soll die Klasse `Burner` durch eine Funktion `contains` erweitert werden, welche diese Prüfung vornehmen kann.

Die Funktion erhält hierfür einen Parameter des Typs `const Point&` übergeben, in dem die Position, an die mit der Maus geklickt wurde, an die Funktion übermittelt wird.

Die Funktion prüft dann, ob sich dieser Punkt zwischen der linken oberen Ecke und der rechten unteren Ecke des Rechtecks befindet, das sich aus der Position `ActualPosition` und den

Abmessungen `ActualDimension` ergibt. Das Ergebnis dieser Prüfung wird schließlich als Wahrheitswert (`bool`) zurückgeliefert.

Trennen Sie auch hier wieder zwischen Deklaration der Klasse und der Implementierung der Funktionen: Fügen Sie den Prototyp der neuen Funktion in die Klassendefinition in der Headerdatei ein und implementieren Sie dann die Funktion in der `.cpp`-Datei.

Hinweis: Verwenden Sie für die Implementierung wieder die Operationen, welche die Klasse `Point` bereitstellt. Insbesondere Additionen und Vergleiche sind hier sehr nützlich, um sich unnötige Arbeit und Fehlersuche zu ersparen.

Prüfen Sie, ob sich das Projekt mit diesen Änderungen korrekt und ohne Fehler compilieren lässt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Ergänzen Sie die Klasse `Burner` um eine Mausbedienbarkeit.

4 Erweiterung der Klasse Boiler

Boiler
+ ContentTemperature :double + ContainedVolume :double + Position :Point + Dimension :Point + Name :string – BBurner :Burner*
+ Boiler(Name :string Position :Point Dimension :Point ContainedVolume :double, ContentTemperature :double) + Boiler(Source :Boiler&) + ~Boiler() + addContent(MediaTemperature :double, MediaAmount :double) :void + addHeat(Amount :double) :void + show() :void + contains() :bool + activateHeating (Amount :double) :void + operator=(rhs :?) : Boiler

Auch die Klasse **Boiler** benötigt einige Erweiterungen. Neben der bereits von **Burner** bekannten Schnittstelle zur Mausbedienung fehlt hier zudem noch die Fähigkeit, selbst ein **Burner**-Objekt zu verwalten.

Hierfür ist, wie nebenstehend zu erkennen ist, ein Pointer zur Verwaltung des **Burner**-Objekts sowie auch eine Funktion zur Bedienung des verknüpften Brenners zu ergänzen.

Darüber hinaus wird je eine Ergänzung an den bestehenden Versionen des Konstruktors und des Destruktors sowie ein zusätzlicher Copy-Konstruktor und ein Zuweisungsoperator erforderlich. Beim Zuweisungsoperator ist dabei noch der korrekte Typ des rechten Operanden zu klären.

Die notwendigen Details zu allen diesen notwendigen Erweiterungen finden Sie in den folgenden Unterabschnitten.

4.1 Integration des Brenners in den Kessel

Damit das **Boiler**-Objekt ein **Burner**-Objekt verwalten kann, müssen Sie ein neues Datenelement in die bereits bestehende Klassendefinition von **Boiler** einfügen, das einen Pointer auf den verwalteten Brenner speichert. Es soll **BBurner** heißen und vom Typ **Burner*** sein.



Integrieren Sie den Brenner in den Kessel.

4.1.1 Konstruktoren und Destruktoren

Dieses neue Datenelement **BBurner** muss selbstverständlich im Konstruktor initialisiert werden.

Hierfür setzen Sie in der Initialisiererliste den **new**-Operator ein, um den Pointer auf ein neu generiertes Objekt zeigen zu lassen. Als Parameter für die Konstruktion dieses neuen **Burner**-Objekts müssen Sie einen Namen für den Brenner (Leerstring) und einen Pointer auf den zu beheizenden Kessel angeben. Der Kessel, den das neue **Burner**-Objekt beheizen soll, ist das soeben in Konstruktion befindliche **Boiler**-Objekt auf das Sie also einen Pointer übergeben müssen.



Erzeugen Sie ein neues Datenelement und erweitern Sie die beiden Konstruktoren entsprechend um eine Initialisierung.

Wichtig: Falls dieser Abschnitt nicht völlig klar war: Ja, das neue Objekt wird während der Konstruktion des **Boiler**-Objekts „zwischen-durch“ konstruiert. Das ist ein ganz normaler Vorgang, z.B. wenn ein

Objekt andere Objekte als Datenelemente enthält. Was auch völlig normal ist.

Das alles macht die Vorgänge bei der Objektkonstruktion oft erstaunlich komplex, aber auch ungemein praktisch: Solange jede Klasse korrekt implementiert ist, greift alles ineinander und „funktioniert ganz einfach“.

Hinweis: Grundsätzlich können Sie in jeder Elementfunktion über den „versteckten“ Parameter **this**, der ein Pointer auf dasjenige Objekt darstellt, von welchem die Funktion gerufen wurde, auf dieses Objekt zugreifen.

Wenn also eine Funktion über **a.foo()** aufgerufen wurde, dann existiert innerhalb dieser Funktion automatisch ein Pointer **this** für den während der Ausführung des Aufrufs **a.foo()** gilt: **this == &a**.

In der Sprache C++ benötigt man diesen **this**-Pointer nicht so häufig explizit wie die entsprechenden Elemente in anderen Sprachen – aber hier liegt ein sehr typischer Anwendungsfall vor: Der Kessel, der den Brenner erzeugt, möchte „seinem“ Brenner gewissermaßen eine „Rückrufnummer“ mitgeben, damit dieser damit zu seinem „Eigentümer“ jederzeit „nach Hause telefonieren“ kann.

Denken Sie daran, dass Sie **zwei** Konstruktoren anpassen müssen. In beiden soll der Pointer mit einem gültigen Pointer auf ein (neues) **Burner**-Objekt gefüllt werden.

Testen Sie, ob Ihr Programm nach dieser Änderung ordnungsgemäß und ohne Fehler compilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Nachdem der Kessel nun Eigentümer des Brenners ist, und diesen verwaltet, muss auch sichergestellt sein, dass das **Burner**-Objekt beim Ende des Lebenszyklusses des Kessels nicht verwaist. Stellen Sie sicher, dass der Destruktor das **Burner**-Objekt korrekt löscht.

Testen Sie, ob Ihr Programm nach dieser Änderung ordnungsgemäß und ohne Fehler compilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Immer wenn eine Kopie eines Objekts hergestellt werden muss (also zum Beispiel wenn eine Variable als Wertparameter an eine Funktion übergeben wird), bringt der Compiler automatisch den sogenannten Copy-Konstruktor zum Einsatz.

Sofern Sie (wie bisher) keine eigene Implementierung für diesen Konstruktor angeben, verwendet der Compiler eine Default-Implementierung,

Recherche ☕
this-Pointer



Erweitern Sie den Destruktor der Klasse **Boiler**.



Ergänzen Sie die Klasse **Boiler** um einen Copy-Konstruktor.

Recherche ☕
Copy-Konstruktoren

die sich so verhält, dass alle Datenelemente des zu kopierenden Objekts identisch kopiert werden. Bei den Objekten vom Klassentyp erfolgt die Kopie dabei durch deren eigenen Copy-Konstruktor, bei allen anderen Datentypen (Pointer, zum Beispiel) wird der Inhalt der Datenelemente bitgetreu kopiert. Pointer enthalten damit also zum Beispiel nach der Kopie die selbe Zieladresse wie das Datenelement im Originalobjekt.

Dieses Verhalten ist im vorliegenden Fall beim Element **BBurner** unerwünscht, da dann zwei Kessel den selben Brenner für sich beanspruchen würden. Dies würde zu Fehlfunktion und Programmabsturz führen.

Ergänzen Sie die daher die Klasse **Boiler** um eine eigene Implementierung eines Copy-Konstruktors. Dieser soll ohne ausführbaren Teil auskommen und alle seine Aktivitäten über die Initialisiererliste regeln.

Dabei sollen alle Datenelemente bis auf **BBurner** mit dem Wert des entsprechenden Datenelements des Originalobjekts initialisiert werden. Lediglich **BBurner** soll mit einem Pointer auf eine **Kopie** des **Burner**-Objekts des Originalobjekts beschrieben werden. Hierfür müssen Sie ein neues Objekt als Kopie dieses **Burner**-Objekts erstellen.

Hinweis: Fertigen Sie eine kleine Skizze an, welche den eben erklärten Sachverhalt zeigt. Die Zeichnung soll Ihnen helfen herauszufinden, was Sie zu tun haben!

Hinweis: Dabei können Sie die Information von oben verwenden, dass für jede Klasse immer ein Copy-Konstruktor existiert – auch wenn sie diesen nicht selbst definiert haben!

Testen Sie, ob Ihr Projekt mit dem neuen Copy-Konstruktor fehlerfrei und ohne Warnungen compilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Fertigen Sie eine Skizze an, welche Aufgabe bei der Erstellung der Kopie zu leisten ist.

4.1.2 Ein Zuweisungsoperator

Ähnlich zum Copy-Konstruktor wird auch beim Zuweisungsoperator der Zustand eines Objekts in ein anderes dupliziert. Dennoch handelt es sich beim Zuweisungsoperator nicht um einen Konstruktor, so dass beide auch sehr unterschiedliche Aufgaben haben.

Frage: Welchen Typ muss der rechte Operand des Zuweisungsoperators besitzen, der einem **Boiler**-Objekt den Wert eines anderen **Boiler**-Objekts zuweist?

Frage: Welche Aufgaben muss dieser Zuweisungsoperator erfüllen?

Implementieren Sie den Zuweisungsoperator gemäß Ihrer Überlegungen.



Implementieren Sie den Zuweisungsoperator:

Recherche ☕☕

Zusammenhang zwischen Copy-Konstruktoren, Zuweisungsoperatoren und Destruktoren

Testen Sie, ob Ihr Programm nach dieser Änderung ordnungsgemäß und ohne Fehler compilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4.1.3 Aktivierungsfunktion für die Heizung

Damit Kessel und Brenner für „Außenstehende“ als Einheit verwendbar werden, fügen Sie eine Funktion `activateHeating` zur Klasse `Boiler` hinzu. Diese soll die Funktion `feedFuel` „ihres“ Brenners aufrufen und dabei den Parameter `Amount` weiterreichen.

Testen Sie, ob Ihr Projekt mit der neuen Funktion fehlerfrei und ohne Warnungen compilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Erzeugen Sie die neue Funktion `activateHeating`.

4.2 Interface für Mausbedienung

Wie bereits bei der Klasse `Burner` soll auch bei der Klasse `Boiler` eine Mausbedienbarkeit ergänzt werden. Hierfür ist analog zum Vorgehen bei `Burner` eine Funktion `contains` zu implementieren.

Allerdings soll diese Zusätzlich zum Klick auf die Fläche des Kessels auf dem Bildschirm auch auf einen Klick auf den Brenner `true` zurückliefern. Daher müssen Sie die Funktion `contains` des `Burner`-Objekts des Kessels aufrufen und das Ergebnis mit in den Rückgabewert der Funktion integrieren.

Prüfen Sie, ob Ihr Projekt mit der neuen Funktion korrekt und ohne Fehlermeldungen compiliert.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Ergänzen Sie die Klasse `Boiler` um eine Mausbedienung.

4.3 Erweiterung von `Boiler::show`

Um zu erreichen, dass beim Zeichnen eines `Boiler`-Objekts auf den Bildschirm auch das damit verbundene `Burner`-Objekt dargestellt wird, muss die Funktion `show()` der Klasse `Boiler` um einen Aufruf der Funktion `show()` des mit dem Kessel verbundenen Brenners ergänzt werden.

Nehmen Sie diese Erweiterung vor.



Erweitern Sie die Funktion `Boiler::show` um einen Aufruf von `show` des verbundenen Brenners.

Taste	Aktion
	Befüllen von Kessel 1 nach Vorwahl
	befüllen von Kessel 2 nach Vorwahl
	Zuführen von Wärme zum Kessel 1 (magisch)
	Zuführen von Wärme zum Kessel 2 (magisch)
	Aktivieren Heizung Kessel 1 (Brenner)
	Aktivieren Heizung Kessel 2 (Brenner)
	Vorwahl des Volumens
	Vorwahl der Temperatur

Tabelle 1: Tastaturbelegung, die für den Testbetrieb im aktuellen Modus `STAGE == INTEGRATE_COMPONENTS` gültig ist.

Prüfen Sie, ob Ihr Projekt mit der neuen Funktion korrekt und ohne Fehlermeldungen compiliert.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

5 Test der integrierten Komponenten

Ändern Sie nun in `Stages.h` die Definition der Präprozessorkonstante `STAGE` auf `STAGE INTEGRATE_COMPONENTS` ab und testen Sie Ihre neuen Implementierungen und beheben Sie eventuelle Fehler. Die Ausgabe auf dem Bildschirm sollte aussehen, wie in Abbildung 2. Sie können dabei die Tastaturkommandos aus Tabelle 1 einsetzen.

Darüber hinaus sollten Sie auch die Bedienung der Brenner mit der Maus testen. Diese sollten aktiviert werden, wenn Sie auf den Kessel oder den

?

Testen Sie die neu entwickelten Funktionen im Modus `INTEGRATE_COMPONENTS` und beheben Sie eventuelle Fehler.

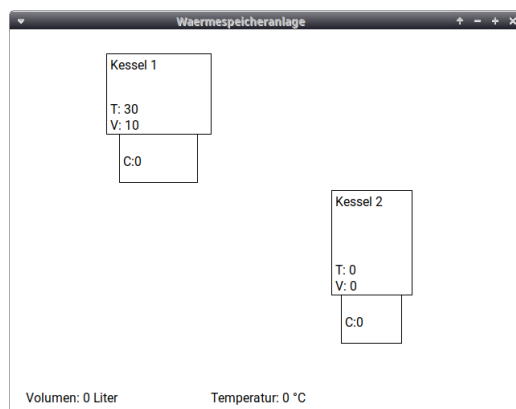


Abbildung 2: Bildschirmdarstellung Programms im neuen Testmodus `STAGE INTEGRATE_COMPONENTS` unmittelbar nach dem Programmstart.

Brenner klicken. Bedenken Sie dabei, dass es physikalisch nicht sinnvoll ist, einem leeren Kessel Wärme zuzuführen.

Frage: Beobachten Sie, wie Ihr Programm auf eine derartige Fehlbedienung reagiert!

Hinweis: Schreiben Sie zu diesem Zeitpunkt keinen Programmcode, um diesen Fehlerfall abzufangen.

Analysieren Sie den durch `STAGE == INTEGRATE_COMPONENTS` aktivierten Testcode in `main.cpp` und klären Sie eventuelle Fragen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

6 Eine Containerklasse für Boiler

Offensichtlich ist das Handhaben der einzelnen Instanzen von `Boiler` ausgesprochen umständlich, wenn die Anlage umfangreicher wird und aus einer größeren Anzahl von Kessel-Brenner-Einheiten besteht.

Damit wird der Programmcode in `main.cpp` sehr aufwendig und unflexibel. Stellen Sie sich z.B. vor, Ihr Programm sollte es dem Bedienpersonal gestatten, zur Laufzeit (also ohne das Programm neu zu compilieren) neue Kessel-Brenner-Einheiten zu erzeugen oder auch zu löschen.

6.1 Die Grundlagen der Containerklasse

StorageComponentContainer
+ ContentList :Boiler** + MaxSize :int + CurrentSize :int
+ StorageComponentContainer (MaxSize :int) + ~StorageComponentContainer() + add(NewOne :Boiler*) :void + getNr() :int + operator[](index :int) :Boiler*

Um diesem Missstand abzuhelpen, soll nun eine Klasse entwickelt werden, welche alle `Boiler`-Objekte einer Anlage verwalten und damit die Programmierung von z.B. Schleifen über eine variable Anzahl an Kesseln unterstützen kann.

Fügen Sie in Ihr Projekt eine neue Headerdatei mit dem Namen `StorageComponentContainer.h` ein und legen Sie in dieser Datei eine neue Klasse `StorageComponentContainer` an.

`StorageComponentContainer` an.

Hinweis: Es ist sinnvoll, dass Sie sich die Verhältnisse anhand einer Zeichnung verdeutlichen, damit sie nicht bei all diesen Pointern gedanklich vom Weg abkommen!

Diese Klasse soll die Datenelemente gemäß folgender Tabelle enthalten:



Entwickeln Sie eine Containerklasse zur Verwaltung der Brenner

Attribut	Typ	Beschreibung
<code>ContentList</code>	<code>Boiler**</code>	Pointer auf ein dynamisch angelegtes Array, das seinerseits Pointer auf die einzelnen verwalteten Kessel enthält
<code>MaxSize</code>	<code>int</code>	Gibt die momentan mögliche Maximalgröße der Liste zur Verwaltung der Kessel an. Diese Zahl muss stets identisch sein zur Anzahl der Elemente des Arrays auf welches der Pointer <code>ContentList</code> zeigt.
<code>CurrentSize</code>	<code>int</code>	Anzahl der genutzten Einträge in <code>ContentList</code> . Diese muss stets kleiner oder gleich der Maximalzahl <code>MaxSize</code> sein. (Klar, denn mehr Einträge hat das Array ja nicht!)

Frage: Das Verhältnis der beiden Datenelemente der Klasse `MaxSize` und `CurrentSize` zueinander ist nicht ganz einfach aus dem Text zu verstehen. Fertigen Sie sich also eine Skizze an und verdeutlichen Sie sich dabei folgende Situationen und die entsprechenden Werte, welche die beiden Variablen annehmen:

- Ein leerer Container bevor das erste Element hinzugefügt wird;
- Ein teilweise gefüllter Container;
- Ein voller Container, der keine weiteren Elemente mehr aufnehmen kann;

Wohin „zeigen“ die beiden Indizes jeweils? Überlegen Sie sich auch, wie Sie die entsprechenden Situationen im Programm ggf. erkennen können.

Nun werden in der Containerklasse noch diverse Funktionen benötigt, die Sie ausnahmsweise direkt in der Klassendefinition (anstelle der Funktionsprototypen) implementieren sollen.

Wichtig: Bei der Klasse `StorageComponentContainer` nehmen Sie die Implementierung der Funktionen *ausnahmsweise* direkt in der Klassendefinition vor. Statt des `;`-Zeichens nach der Parameterliste der Funktion folgt an in diesem Fall am dessen Stelle ein Anweisungsblock `{ ... }` mit der Implementierung bzw. die Initialisiererliste bei den Konstruktoren.

Implementieren Sie zunächst die Konstruktoren und Destruktoren:

Methode	Typ	Beschreibung
Konstruktor	—	Der Konstruktor soll ein optionales (Defaultwert 10) Argument <code>MaxSize</code> vom Typ <code>int</code> erhalten. Der Wert dieses Parameters soll in das entsprechende Datenelement kopiert werden. Das Element <code>CurrentSize</code> soll mit 0 initialisiert werden. Schließlich soll noch (mit <code>new</code>) ein neues Array der passenden Größe angelegt und der Pointer darauf in <code>ContentList</code> gespeichert werden.
Destruktor	—	Hier sollen alle verwalteten Brenner-Objekte und anschließend auch das von <code>ContentList</code> referenzierte Array gelöscht (mit <code>delete</code>) werden.

Wichtig: Zur Erinnerung: Ein Destruktor der Klasse löscht die vom Objekt als **Eigentum** beanspruchten **fremden** Objekte. Versuchen Sie keinesfalls nochmals den bereits laufenden Löschvorgang des `this`-Objekts erneut zu initiieren oder Datenelemente der Klasse zu löschen.

Hinweis: Die Methoden in der folgenden Tabelle bewirken genau das, was man trivialerweise annehmen würde. Zusätzliche Fehlerüberprüfungen können (und sollen) Sie nach Belieben ergänzen.

Die Klasse soll außerdem folgende Funktionen anbieten:

Methoden	Typ	Beschreibung
<code>add</code>	<code>void</code>	Die Methode soll einen Parameter vom Typ <code>Boiler*</code> besitzen und diesen Pointer in die <code>ContentList</code> an der nächsten freien Stelle aufnehmen. <code>CurrentSize</code> ist dabei zu inkrementieren.
<code>operator[]</code>	<code>Boiler*</code>	Liefert den Pointer, der in dem Element des Arrays <code>ContentList</code> gespeichert ist, welches den Index hat, der dem Wert des zweiten Operanden (dem zwischen den beiden Klammern) entspricht.
<code>getNr</code>	<code>int</code>	Liefert den Wert von <code>CurrentSize</code> .

6.2 Die Iterator-Schnittstelle

Mit den bisherigen Implementierungen kann man den Container bereits verwenden. So kann beispielsweise Code folgender Form geschrieben werden:

```
StorageComponentContainer c;
...

c.add(new Boiler("Foo", Point(100,100)));
...
for(int i=0; i<c.getNr(); i++)
{
    c[i]->show();
}
```

Allerdings ist dies, insbesondere die Verwendung in der Schleife, etwas sperrig und es sind viele Kenntnisse über die Interna des Containers nötig.

Daher wurde in C++ das Konzept der Iteratoren eingeführt und auch bei der Implementierung der Standardbibliothek großflächig eingesetzt. Die Grundidee ist dabei folgende:

Eine Klasse, die das Iterieren über mehrere Objekte unterstützen soll, erhält die Funktionen `begin` und `end`, welche jeweils ein „Iterator“-Objekt zurückliefern. Diese Iterator-Objekte „wissen“, wie die Nachbarobjekte im Container zu finden sind und „können“ dorthin navigieren².

²Die Standardbibliothek kennt dabei unterschiedlich „mächtige“ Iteratoren: Manche können nur inkrementieren, manche können auch dekrementieren und wieder andere können sogar um `n` Objekte springen oder gezielt das `n`-te Objekt aufsuchen. Hier in dieser Aufgabe wird von Ihnen eine reine Inkrement-Iteratorklasse entwickelt.



Ergänzen Sie ein Iterator-Interface für Ihre Containerklasse.

Recherche ☕☕☕

Iteratoren: Fähigkeiten und Funktion

Erzeugen Sie in der bereits bestehenden Headerdatei `StorageComponentContainer.h` eine weitere Klasse. Diese neu anzulegende Klasse soll den Namen `StorageComponentContainerIterator` tragen und **vor** der Klasse `StorageComponentContainer` in der Quelldatei definiert werden.

Da diese neue Klasse allerdings ihrerseits auch Pointer auf Objekte der Klasse `StorageComponentContainer` verarbeiten muss, benötigt sie vor ihrer eigenen Definition unbedingt eine Kurzdeklaration der Klasse `StorageComponentContainer`.

StorageComponentContainerIterator
+ ItemPointer :Boiler**
+ StorageComponentContainerIterator (Item :Boiler**)
+ operator++()
:StorageComponentContainerIterator
+ operator++(x :int)
:StorageComponentContainerIterator
+ operator==(rhs
:StorageComponentContainerIterator)
:bool
+ operator!=(rhs
:StorageComponentContainerIterator)
:bool
+ operator*() :Boiler&
+ operator->() :Boiler*

Die neue Iteratorklasse ist im Prinzip sehr einfach, aber aufgrund ihrer besonderen Aufgabe sind einige Dinge etwas ungewohnt.

Eher einfach ist das Datenelement: Es speichert einen Pointer auf ein Element des Arrays `ContentList` in einem Objekt Ihrer Containerklasse. Da es sich in diesem Fall bei den Elementen um Pointer handelt, ist hier als Typ ein Pointer auf einen anderen Pointer nötig, der dann auf das gesuchte Objekt im Container verweist.

Hinweis: Verdeutlichen Sie sich diesen Zusammenhang anhand einer Zeichnung!

Weitere Datenelemente benötigt diese Iteratorklasse nicht. Allerdings sind noch einige Funktionen und Operatoren zu implementieren:

Konstruktor: Der Konstruktor initialisiert das Datenelement mit dem übergebenen Parameter.

operator++: Um einen Pointer auf das nächste Element im Array `ContentList` zu erhalten, muss der Pointer `Itempointer` mit `++` inkrementiert werden, da die Pointer im Array ohne Lücken gespeichert werden. Der Rückgabewert ist aber nicht der Pointer sondern (wie üblich) das `*this`-Objekt.

Hinweis: Sollte das alles nicht glasklar sein: Fertigen Sie sich wie üblich eine Skizze an, mit der Sie sich die Situation verdeutlichen. Diskutieren Sie das Thema mit Ihren Betreuern.

operator++(int): Für das Post-Inkrement gilt im wesentlichen das Selbe wie für das Pre-Inkrement. Allerdings muss (auch wie üblich) vor dem Inkrementieren des Pointers eine Kopie des `*this`-Objekts erzeugt werden, um es als Rückgabewert verwenden zu können.

operator==: Zwei Iteratorobjekte sollen gleich sein, wenn sie an die selbe Adresse im Array `ContentList` verweisen.

operator!=: Zwei Iteratorobjekte sollen ungleich sein, wenn sie nicht an die selbe Adresse im Array `ContentList` verweisen.

operator*: Der unäre Operator `*`, auch Dereferenzierungsoperator genannt, soll das vom Container referenzierte Objekt zurückliefern. Also muss der Pointer `Itempointer` nicht nur einmal, sondern **zweimal** dereferenziert werden.

operator->: Der „Pfeil“-Operator `->` wird von uns hier zum ersten Mal implementiert. Er ist letztlich ja nur eine Variante des Dereferenzierungsoperators und muss auch implementiert werden, damit er verwendet werden kann.

Hier ist der Pointer `Itempointer` **einmal** dereferenziert zurückzuliefern, da als Rückgabewert vom Compiler ein Pointer auf das Zielobjekt erwartet wird, nicht das Zielobjekt selbst wie beim gewöhnlichen Dereferenzierungsoperator `*`.

Damit ist die Iteratorklasse fertig und kann zur Ergänzung der Iteratorschnittstelle der Klasse `StorageComponentContainer` verwendet werden.

StorageComponentContainer
+ ContentList :Boiler** + MaxSize :int + CurrentSize :int
+ StorageComponentContainer (MaxSize :int) + ~StorageComponentContainer() + add(NewOne :Boiler*) :void + getNr() :int + operator[](index :int) :Boiler*& + begin() :StorageComponentContainerIterator + end() :StorageComponentContainerIterator

Hierfür sind die beiden neuen Methoden `begin` und `end` zu implementieren. Diese dienen dazu, die Iteratorobjekte anderen Codeteilen zur Verfügung zu stellen.

Implementieren Sie alle Funktionen entsprechend der Spezifikation bevor Sie mit dem dynamischen Test im nächsten Abschnitt fortfahren.

Zur Erzeugung der entsprechenden Iteratorobjekte für die Rückgabewerte der Methoden können Sie wie gewohnt die Parameter der Konstruktoren verwenden.

Methode	Beschreibung
<code>begin()</code>	Liefert einen Iterator (Typ: <code>StorageComponentContainerIterator</code>), der das erste Element im Container referenziert;
<code>end()</code>	Liefert einen Iterator (Typ: <code>StorageComponentContainerIterator</code>), der das Element nach dem letzten Element im Container referenziert; Verwenden Sie dafür tatsächlich den Pointer auf das nicht mehr vorhandene Objekt, das hinter dem letzten verfügbaren Objekt liegen würde.

7 Test

Ändern Sie nun in `Stages.h` die Definition der Präprozessorkonstante `STAGE` auf `STAGE_CREATE_CONTAINER` ab und testen Sie Ihre neuen Im-

?

Testen Sie die neu entwickelten Funktionen im Modus `CREATE_CONTAINER` und beheben Sie eventuelle Fehler.

Taste	Aktion
ctrl a ... ctrl d	Aktivieren der Heizung 1-4
1 ... 9	Vorwahl des Volumens
A ... Z	Vorwahl der Temperatur
a ... d	Befüllen Kessel 1..4 nach Vorwahl

Tabelle 2: Tastaturbelegung, die für den Testbetrieb im aktuellen Modus `STAGE == CREATE_CONTAINER` gültig ist.

plementierungen und beheben Sie eventuelle Fehler. Die Ausgabe auf dem Bildschirm sollte aussehen, wie in Abbildung 3. Sie können dabei die Tastaturkommandos aus Tabelle 2 einsetzen.

Darüber hinaus sollten Sie auch die Bedienung der Brenner mit der Maus testen. Diese sollten aktiviert werden, wenn Sie auf den Kessel oder den Brenner klicken.

Analysieren Sie den durch `STAGE == CREATE_CONTAINER` aktivierten Testcode in `main.cpp` und klären Sie eventuelle Fragen.



Analysieren Sie den Testcode der bei `CREATE_CONTAINER` neu aktiviert wird.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Die Entwicklung an der Wärmespeicheranlage wird in der nächsten Aufgabe fortgeführt.

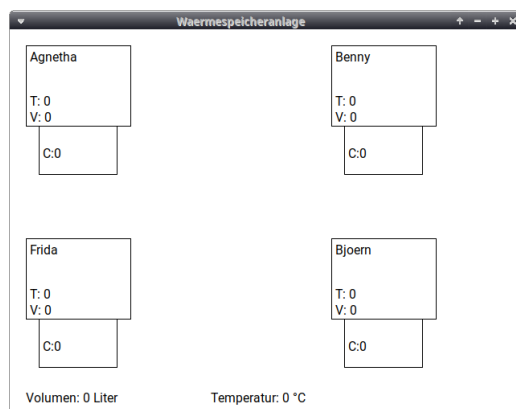


Abbildung 3: Bildschirmdarstellung Programms im neuen Testmodus `STAGE CREATE_CONTAINER` unmittelbar nach dem Programmstart.